

delivered before sending m^* . P_3 therefore decides to postpone the delivery of m^* (and will also not adjust its local, logical clock). Later, after m has been received and delivered by P_3 , which brings its local clock to $(1, 0, 0)$, P_3 can deliver message m^* and also update its clock.

A note on ordered message delivery. Some middleware systems, notably ISIS and its successor Horus [Birman and van Renesse, 1994], provide support for total-ordered and causal-ordered (reliable) multicasting. There has been some controversy whether such support should be provided as part of the message-communication layer, or whether applications should handle ordering (see, e.g., Cheriton and Skeen [1993]; Birman [1994]). Matters have not been settled, but more important is that the arguments still hold today.

There are two main problems with letting the middleware deal with message ordering. First, because the middleware cannot tell what a message actually contains, only *potential* causality is captured. For example, two messages from the same sender that are completely independent will always be marked as causally related by the middleware layer. This approach is overly restrictive and may lead to efficiency problems.

A second problem is that not all causality may be captured. Consider an electronic bulletin board. Suppose Alice posts an article. If she then phones Bob telling about what she just wrote, Bob may post another article as a reaction without having seen Alice's posting on the board. In other words, there is a causality between Bob's posting and that of Alice due to *external* communication. This causality is not captured by the bulletin board system.

In essence, ordering issues, like many other application-specific communication issues, can be adequately solved by looking at the application for which communication is taking place. This is also known as the **end-to-end argument** in systems design [Saltzer et al., 1984]. A drawback of having only application-level solutions is that a developer is forced to concentrate on issues that do not immediately relate to the core functionality of the application. For example, ordering may not be the most important problem when developing a messaging system such as an electronic bulletin board. In that case, having an underlying communication layer handle ordering may turn out to be convenient. We will come across the end-to-end argument a number of times.

6.3 Mutual exclusion

Fundamental to distributed systems is the concurrency and collaboration among multiple processes. In many cases, this also means that processes will need to simultaneously access the same resources. To prevent that such concurrent accesses corrupt the resource, or make it inconsistent, solutions are needed to grant mutual exclusive access by processes. In this section, we take a look at some important and representative distributed algorithms that have been proposed. Surveys of distributed algorithms for mutual exclusion are

provided by Saxena and Rai [2003] and Velazquez [1993]. Various algorithms are also presented in [Kshemkalyani and Singhal, 2008].

Overview

Distributed mutual exclusion algorithms can be classified into two different categories. In **token-based solutions** mutual exclusion is achieved by passing a special message between the processes, known as a **token**. There is only one token available and who ever has that token is allowed to access the shared resource. When finished, the token is passed on to a next process. If a process having the token is not interested in accessing the resource, it passes it on.

Token-based solutions have a few important properties. First, depending on how the processes are organized, they can fairly easily ensure that every process will get a chance at accessing the resource. In other words, they avoid **starvation**. Second, **deadlocks** by which several processes are indefinitely waiting for each other to proceed, can easily be avoided, contributing to their simplicity. The main drawback of token-based solutions is a rather serious one: when the token is lost (e.g., because the process holding it crashed), an intricate distributed procedure needs to be started to ensure that a new token is created, but above all, that it is also the only token.

As an alternative, many distributed mutual exclusion algorithms follow a **permission-based approach**. In this case, a process wanting to access the resource first requires the permission from other processes. There are many different ways toward granting such a permission and in the sections that follow we will consider a few of them.

A centralized algorithm

A straightforward way to achieve mutual exclusion in a distributed system is to simulate how it is done in a one-processor system. One process is elected as the coordinator. Whenever a process wants to access a shared resource, it sends a request message to the coordinator stating which resource it wants to access and asking for permission. If no other process is currently accessing that resource, the coordinator sends back a reply granting permission, as shown in Figure 6.15(a). When the reply arrives, the requester can go ahead.

Now suppose that another process, P_2 in Figure 6.15(b) asks for permission to access the resource. The coordinator knows that a different process is already at the resource, so it cannot grant permission. The exact method used to deny permission is system dependent. In Figure 6.15(b) the coordinator just refrains from replying, thus blocking process P_2 , which is waiting for a reply. Alternatively, it could send a reply saying “permission denied.” Either way, it queues the request from P_2 for the time being and waits for more messages.

When process P_1 is finished with the resource, it sends a message to the coordinator releasing its exclusive access, as shown in Figure 6.15(c). The

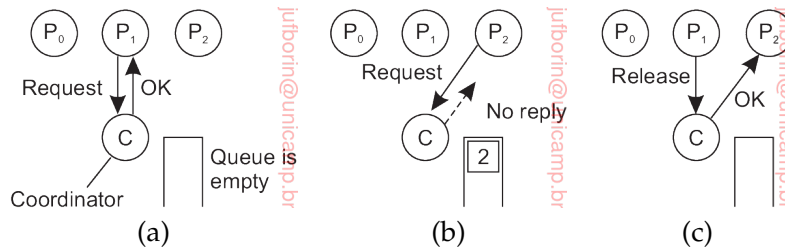


Figure 6.15: (a) Process P_1 asks for permission to access a shared resource. Permission is granted. (b) Process P_2 asks permission to access the same resource, but receives no reply. (c) When P_1 releases the resource, the coordinator replies to P_2 .

coordinator takes the first item off the queue of deferred requests and sends that process a grant message. If the process was still blocked (i.e., this is the first message to it), it unblocks and accesses the resource. If an explicit message has already been sent denying permission, the process will have to poll for incoming traffic or block later. Either way, when it sees the grant, it can go ahead as well.

It is easy to see that the algorithm guarantees mutual exclusion: the coordinator lets only one process at a time access the resource. It is also fair, since requests are granted in the order in which they are received. No process ever waits forever (no starvation). The scheme is easy to implement, too, and requires only three messages per use of resource (request, grant, release). Its simplicity makes it an attractive solution for many practical situations.

The centralized approach also has shortcomings. The coordinator is a single point of failure, so if it crashes, the entire system may go down. If processes normally block after making a request, they cannot distinguish a dead coordinator from “permission denied” since in both cases no message comes back. In addition, in a large system, a single coordinator can become a performance bottleneck. Nevertheless, the benefits coming from its simplicity outweigh in many cases the potential drawbacks. Moreover, distributed solutions are not necessarily better, as we illustrate next.

A distributed algorithm

Using Lamport’s logical clocks, and inspired by Lamport’s original solution for distributed mutual exclusion (which we discussed in Note 6.3), Ricart and Agrawala [1981] provided the following algorithm. Their solution requires a total ordering of all events in the system. That is, for any pair of events, such as messages, it must be unambiguous which one actually happened first.

The algorithm works as follows. When a process wants to access a shared resource, it builds a message containing the name of the resource, its process

number, and the current (logical) time. It then sends the message to all other processes, conceptually including itself. The sending of messages is assumed to be reliable; that is, no message is lost.

When a process receives a request message from another process, the action it takes depends on its own state with respect to the resource named in the message. Three different cases have to be clearly distinguished:

- If the receiver is not accessing the resource and does not want to access it, it sends back an OK message to the sender.
- If the receiver already has access to the resource, it simply does not reply. Instead, it queues the request.
- If the receiver wants to access the resource as well but has not yet done so, it compares the timestamp of the incoming message with the one contained in the message that it has sent everyone. The lowest one wins. If the incoming message has a lower timestamp, the receiver sends back an OK message. If its own message has a lower timestamp, the receiver queues the incoming request and sends nothing.

After sending out requests asking permission, a process sits back and waits until everyone else has given permission. As soon as all the permissions are in, it may go ahead. When it is finished, it sends OK messages to all processes in its queue and deletes them all from the queue. If there is no conflict, it clearly works. However, suppose that two processes try to simultaneously access the resource, as shown in Figure 6.16(a).

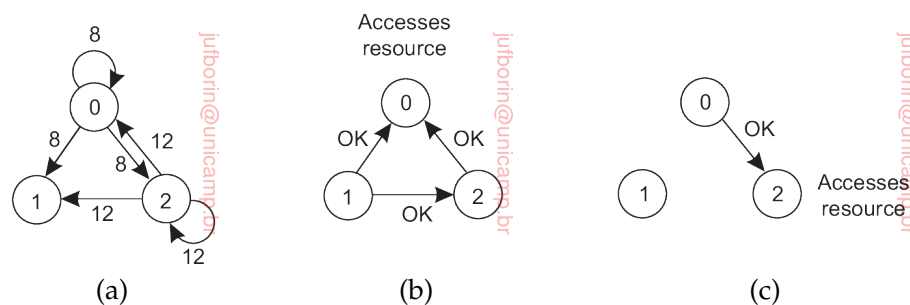


Figure 6.16: (a) Two processes want to access a shared resource at the same moment. (b) P₀ has the lowest timestamp, so it wins. (c) When process P₀ is done, it sends an OK also, so P₂ can now go ahead.

Process P₀ sends everyone a request with timestamp 8, while at the same time, process P₂ sends everyone a request with timestamp 12. P₁ is not interested in the resource, so it sends OK to both senders. Processes P₀ and P₂ both see the conflict and compare timestamps. P₂ sees that it has lost, so it grants permission to P₀ by sending OK. Process P₀ now queues the

request from P_2 for later processing and accesses the resource, as shown in Figure 6.16(b). When it is finished, it removes the request from P_2 from its queue and sends an OK message to P_2 , allowing the latter to go ahead, as shown in Figure 6.16(c). The algorithm works because in the case of a conflict, the lowest timestamp wins and everyone agrees on the ordering of the timestamps.

With this algorithm, mutual exclusion is guaranteed without deadlock or starvation. If the total number of processes is N , then the number of messages that a process needs to send and receive before it can enter its critical section is $2 \cdot (N - 1)$: $N - 1$ request messages to all other processes, and subsequently $N - 1$ OK messages, one from each other process.

Unfortunately, this algorithm has N points of failure. If any process crashes, it will fail to respond to requests. This silence will be interpreted (incorrectly) as denial of permission, thus blocking all subsequent attempts by all processes to enter any of their respective critical regions. The algorithm can be patched up as follows. When a request comes in, the receiver always sends a reply, either granting or denying permission. Whenever either a request or a reply is lost, the sender times out and keeps trying until either a reply comes back or the sender concludes that the destination is dead. After a request is denied, the sender should block waiting for a subsequent OK message.

Another problem with this algorithm is that either a multicast communication primitive must be used, or each process must maintain the group membership list itself, including processes entering the group, leaving the group, and crashing. The method works best with small groups of processes that never change their group memberships. Finally, note that *all* processes are involved in *all* decisions concerning accessing the shared resource, which may impose a burden on processes running on resource-constrained machines.

Various minor improvements are possible to this algorithm. For example, getting permission from everyone is overkill. All that is needed is a method to prevent two processes from accessing the resource at the same time. The algorithm can be modified to grant permission when it has collected permission from a simple majority of the other processes, rather than from all of them.

A token-ring algorithm

A completely different approach to deterministically achieving mutual exclusion in a distributed system is illustrated in Figure 6.17. In software, we construct an overlay network in the form of a logical ring in which each process is assigned a position in the ring. All that matters is that each process knows who is next in line after itself.

When the ring is initialized, process P_0 is given a **token**. The token circulates around the ring. Assuming there are N processes, the token is passed from process P_k to process $P_{(k+1) \bmod N}$ in point-to-point messages. When a process acquires the token from its neighbor, it checks to see if it

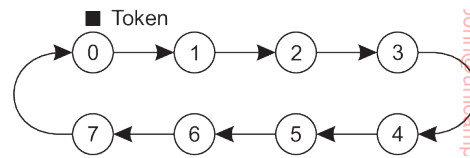


Figure 6.17: An overlay network constructed as a logical ring with a token circulating between its members.

needs to access the shared resource. If so, the process goes ahead, does all the work it needs to, and releases the resources. After it has finished, it passes the token along the ring. It is not permitted to immediately enter the resource again using the same token.

If a process is handed the token by its neighbor and is not interested in the resource, it just passes the token along. As a consequence, when no processes need the resource, the token just circulates around the ring.

The correctness of this algorithm is easy to see. Only one process has the token at any instant, so only one process can actually get to the resource. Since the token circulates among the processes in a well-defined order, starvation cannot occur. Once a process decides it wants to have access to the resource, at worst it will have to wait for every other process to use the resource.

This algorithm has its own problems. If the token is ever lost, for example, because its holder crashes or due to a lost message containing the token, it must be regenerated. In fact, detecting that it is lost may be difficult, since the amount of time between successive appearances of the token on the network is unbounded. The fact that the token has not been spotted for an hour does not mean that it has been lost; somebody may still be using it.

The algorithm also runs into trouble if a process crashes, but recovery is relatively easy. If we require a process receiving the token to acknowledge receipt, a dead process will be detected when its neighbor tries to give it the token and fails. At that point the dead process can be removed from the group, and the token holder can throw the token over the head of the dead process to the next member down the line, or the one after that, if necessary. Of course, doing so requires that everyone maintains the current ring configuration.

A decentralized algorithm

Let us take a look at fully decentralized solution. Lin et al. [2004] propose to use a voting algorithm. Each resource is assumed to be replicated N times. Every replica has its own coordinator for controlling the access by concurrent processes.

However, whenever a process wants to access the resource, it will simply need to get a majority vote from $m > N/2$ coordinators. We assume that when a coordinator does not give permission to access a resource (which it

will do when it had granted permission to another process), it will tell the requester.

The assumption is that when a coordinator crashes, it recovers quickly but will have forgotten any vote it gave before it crashed. Another way of viewing this is that a coordinator resets itself at arbitrary moments. The risk that we are taking is that a reset will make the coordinator forget that it had previously granted permission to some process to access the resource. As a consequence, it may incorrectly grant this permission again to another process after its recovery.

Let $p = \Delta t/T$ be the probability that a coordinator resets during a time interval Δt , while having a lifetime of T . The probability $\mathbb{P}[k]$ that k out of m coordinators reset during the same interval is then

$$\mathbb{P}[k] = \binom{m}{k} p^k (1-p)^{m-k}$$

If f coordinators reset, then the correctness of the voting mechanism will be violated when we have more than m other coordinators think it is okay to allocate the resource, that is, when $N - (m - f) \geq m$, or, in other words, when $f \geq 2m - N$. The probability that such a violation occurs is then $\sum_{k=2m-N}^m \mathbb{P}[k]$. To give an impression of what this could mean, Figure 6.18 shows the probability of violating correctness for different values of N , m , and p . Note that we compute p by considering the number of seconds per hour that a coordinator resets, and also taking this value to be the average time needed to access a resource. Our values for p are considered to be (very) conservative. The conclusion is that, in general, the probability of violating correctness can be so low that it can be neglected in comparison to other types of failure.

TBD: RECOMPUTE NUMBERS DUE TO CHANGE OF VIOLATION FORMULA (SEE MAIL Christian Zirpins)

N	m	p	Violation	N	m	p	Violation
8	5	3 sec/hour	$< 10^{-15}$	8	5	30 sec/hour	$< 10^{-10}$
8	6	3 sec/hour	$< 10^{-18}$	8	6	30 sec/hour	$< 10^{-11}$
16	9	3 sec/hour	$< 10^{-27}$	16	9	30 sec/hour	$< 10^{-18}$
16	12	3 sec/hour	$< 10^{-36}$	16	12	30 sec/hour	$< 10^{-24}$
32	17	3 sec/hour	$< 10^{-52}$	32	17	30 sec/hour	$< 10^{-35}$
32	24	3 sec/hour	$< 10^{-73}$	32	24	30 sec/hour	$< 10^{-49}$

Figure 6.18: Violation probabilities for various parameter values of decentralized mutual exclusion.

To implement this scheme, we can use a system in which a resource is replicated N times. Assume that the resource is known under its unique

name $rname$. We can then assume that the i -th replica is named $rname_i$ which is then used to compute a unique key using a known hash function. As a consequence, every process can generate the N keys given a resource's name, and subsequently look up each node responsible for a replica (and controlling access to that replica) using some commonly used naming system.

If permission to access the resource is denied (i.e., a process gets less than m votes), it is assumed that it will back off for some randomly chosen time, and make a next attempt later. The problem with this scheme is that if many nodes want to access the same resource, it turns out that the utilization rapidly drops. In that case, there are so many nodes competing to get access that eventually no one is able to get enough votes leaving the resource unused. A solution to solve this problem can be found in [Lin et al., 2004].

Note 6.5 (More information: A comparison of the mutual-exclusion algorithms)

Algorithm	Messages per entry/exit	Delay before entry (in message times)
Centralized	3	2
Distributed	$2 \cdot (N - 1)$	$2 \cdot (N - 1)$
Token ring	$1, \dots, \infty$	$0, \dots, N - 1$
Decentralized	$2 \cdot m \cdot k + m, k = 1, 2, \dots$	$2 \cdot m \cdot k$

Figure 6.19: A comparison of four mutual exclusion algorithms.

A brief comparison of the mutual exclusion algorithms we have looked at is instructive. In Figure 6.19 we have listed the algorithms and two performance properties: the number of messages required for a process to access and release a shared resource, and the delay before access can occur (assuming messages are passed sequentially over a network).

In the following, we assume only point-to-point messages (or, equivalently, count a multicast to N processes as N messages).

- The centralized algorithm is simplest and also most efficient. It requires only three messages to enter and leave a critical region: a request, a grant to enter, and a release to exit.
- The distributed algorithm requires $N - 1$ request messages, one to each of the other processes, and an additional $N - 1$ grant messages, for a total of $2 \cdot (N - 1)$.
- With the token ring algorithm, the number is variable. If every process constantly wants to enter a critical region, then each token pass will result in one entry and exit, for an average of one message per critical region entered. At the other extreme, the token may sometimes circulate for hours without anyone being interested in it. In this case, the number of messages per entry into a critical region is unbounded.
- In the decentralized case, we see that a request message needs to be sent to m coordinators, followed by a response message. However, it is possible

that several attempts need to be made (for which we introduce the variable k). An exit requires sending a message to each of the m coordinators.

The delay from the moment a process needs to enter a critical region until its actual entry also varies. For a worst-case analysis, we assume that messages are sent one after the other (i.e., there are never two or more messages in transit at the same time), and that message transfer time is roughly the same everywhere. Delay can then be expressed in **message transfer time units**, or simply **MTTU**. Under these assumptions, when the time using a resource is short, the dominant factor in the delay is determined by the total number of messages sent through the system before access can be granted. When resources are used for a long period of time, the dominant factor is waiting for everyone else to take their turn. In Figure 6.19 we show the former case.

- It takes only two MTTUs to enter a critical region in the centralized case, caused by a request message and the subsequent grant message sent by the coordinator.
- The distributed algorithm requires sending $N - 1$ request messages, and receiving another $N - 1$ grant messages, adding up to $2 \cdot (N - 1)$ MTTUs.
- For the token ring, the delay varies from 0 MTTU (in case the token had just arrived) to $N - 1$ (for when the token had just departed).
- The decentralized case requires sending m messages to coordinators, and another m responses, but a process may need to go through $k \geq 1$ attempts, adding up to $2 \cdot m \cdot k$ MTTUs.

Virtually all algorithms suffer badly in the event of crashes. Special measures and additional complexity must be introduced to avoid having a crash bring down the entire system. It is somewhat ironic that distributed algorithms are generally more sensitive to crashes than centralized ones. In this sense, it should not come as a surprise that, indeed, centralized mutual exclusion is widely applied: it is simple to understand the behavior, and relatively easy to increase the fault tolerance of the centralized server. However, centralized solutions may suffer from scalability problems.

6.4 Election algorithms

Many distributed algorithms require one process to act as coordinator, initiator, or otherwise perform some special role. In general, it does not matter which process takes on this special responsibility, but one of them has to do it. In this section we will look at algorithms for electing a coordinator (using this as a generic name for the special process).

If all processes are exactly the same, with no distinguishing characteristics, there is no way to select one of them to be special. Consequently, we will assume that each process P has a unique identifier $id(P)$. In general, election algorithms attempt to locate the process with the highest identifier and

designate it as coordinator. The algorithms differ in the way they locate the coordinator.

Furthermore, we also assume that every process knows the identifier of every other process. In other words, each process has complete knowledge of the process group in which a coordinator must be elected. What the processes do not know is which ones are currently up and which ones are currently down. The goal of an election algorithm is to ensure that when an election starts, it concludes with all processes agreeing on who the new coordinator is to be. There are many algorithms and variations, of which several important ones are discussed in the text books by Tel [2000] and Lynch [1996].

The bully algorithm

A well-known solution for electing a coordinator is the **bully algorithm** devised by Garcia-Molina [1982]. In the following, we consider N processes $\{P_0, \dots, P_{N-1}\}$ and let $id(P_k) = k$. When any process notices that the coordinator is no longer responding to requests, it initiates an election. A process, P_k , holds an election as follows:

1. P_k sends an ELECTION message to all processes with higher identifiers: $P_{k+1}, P_{k+2}, \dots, P_{N-1}$.
2. If no one responds, P_k wins the election and becomes coordinator.
3. If one of the higher-ups answers, it takes over and P_k 's job is done.

At any moment, a process can get an ELECTION message from one of its lower-numbered colleagues. When such a message arrives, the receiver sends an OK message back to the sender to indicate that he is alive and will take over. The receiver then holds an election, unless it is already holding one. Eventually, all processes give up but one, and that one is the new coordinator. It announces its victory by sending all processes a message telling them that starting immediately it is the new coordinator.

If a process that was previously down comes back up, it holds an election. If it happens to be the highest-numbered process currently running, it will win the election and take over the coordinator's job. Thus the biggest guy in town always wins, hence the name "bully algorithm."

In Figure 6.20 we see an example of how the bully algorithm works. The group consists of eight processes, with identifiers numbered from 0 to 7. Previously process P_7 was the coordinator, but it has just crashed. Process P_4 is the first one to notice this, so it sends ELECTION messages to all the processes higher than it, namely P_5 , P_6 , and P_7 , as shown in Figure 6.20(a). Processes P_5 and P_6 both respond with OK, as shown in Figure 6.20(b). Upon getting the first of these responses, P_4 knows that its job is over, knowing that either one of P_5 or P_6 will take over and become coordinator. Process P_4 just

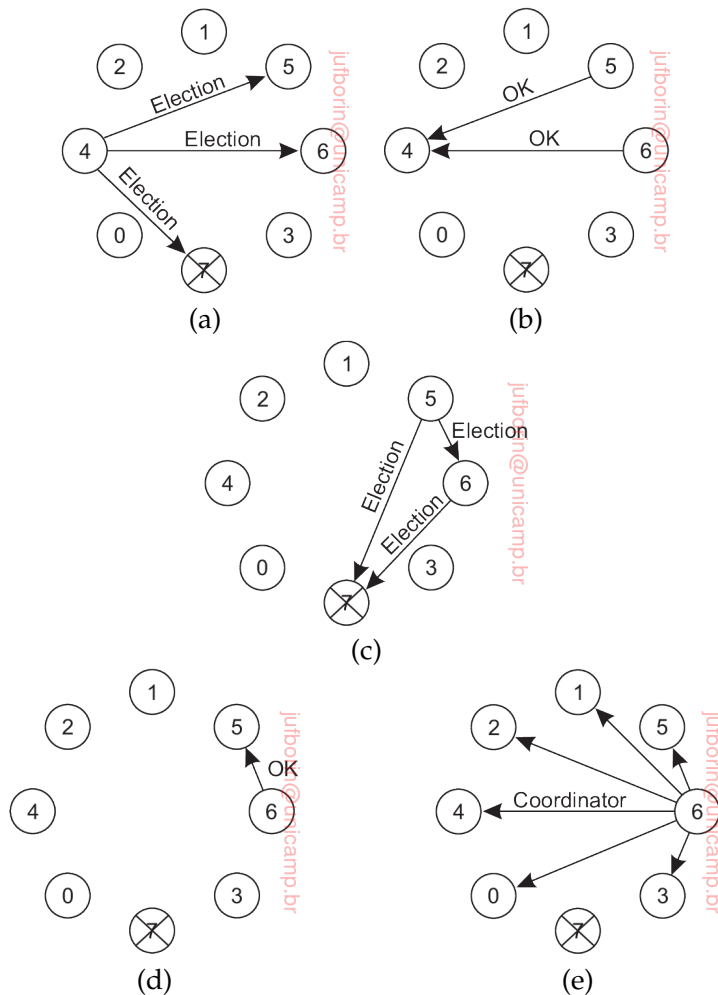


Figure 6.20: The bully election algorithm. (a) Process 4 holds an election. (b) Processes 5 and 6 respond, telling 4 to stop. (c) Now 5 and 6 each hold an election. (d) Process 6 tells 5 to stop. (e) Process 6 wins and tells everyone.

sits back and waits to see who the winner will be (although at this point it can make a pretty good guess).

In Figure 6.20(c) both P_5 and P_6 hold elections, each one sending messages only to those processes with identifiers higher than itself. In Figure 6.20(d), P_6 tells P_5 that it will take over. At this point P_6 knows that P_7 is dead and that it (P_6) is the winner. If there is state information to be collected from disk or elsewhere to pick up where the old coordinator left off, P_6 must now do what is needed. When it is ready to take over, it announces the takeover by sending a COORDINATOR message to all running processes. When P_4 gets

this message, it can now continue with the operation it was trying to do when it discovered that P_7 was dead, but using P_6 as the coordinator this time. In this way the failure of P_7 is handled and the work can continue.

If process P_7 is ever restarted, it will send all the others a COORDINATOR message and bully them into submission.

A ring algorithm

Consider the following election algorithm that is based on the use of a (logical) ring. Unlike some ring algorithms, this one does not use a token. We assume that each process knows who its successor is. When any process notices that the coordinator is not functioning, it builds an ELECTION message containing its own process identifier and sends the message to its successor. If the successor is down, the sender skips over the successor and goes to the next member along the ring, or the one after that, until a running process is located. At each step along the way, the sender adds its own identifier to the list in the message effectively making itself a candidate to be elected as coordinator.

Eventually, the message gets back to the process that started it all. That process recognizes this event when it receives an incoming message containing its own identifier. At that point, the message type is changed to COORDINATOR and circulated once again, this time to inform everyone else who the coordinator is (the list member with the highest identifier) and who the members of the new ring are. When this message has circulated once, it is removed and everyone goes back to work.

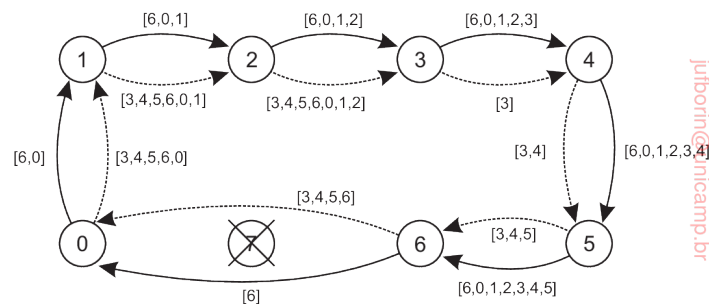


Figure 6.21: Election algorithm using a ring. The solid line shows the election messages initiated by P_6 ; the dashed one those by P_3 .

In Figure 6.21 we see what happens if two processes, P_3 and P_6 , discover simultaneously that the previous coordinator, process P_7 , has crashed. Each of these builds an ELECTION message and each of them starts circulating its message, independent of the other one. Eventually, both messages will go all the way around, and both P_3 and P_6 will convert them into COORDINATOR messages, with exactly the same members and in the same order. When both have gone around again, both will be removed. It does no harm to have extra

messages circulating; at worst it consumes a little bandwidth, but this is not considered wasteful.

Elections in wireless environments

Traditional election algorithms are generally based on assumptions that are not realistic in wireless environments. For example, they assume that message passing is reliable and that the topology of the network does not change. These assumptions are false in most wireless environments, especially those for mobile ad hoc networks.

Only few protocols for elections have been developed that work in ad hoc networks. Vasudevan et al. [2004] propose a solution that can handle failing nodes and partitioning networks. An important property of their solution is that the *best* leader can be elected rather than just a random one as was more or less the case in the previously discussed solutions. Their protocol works as follows. To simplify our discussion, we concentrate only on ad hoc networks and ignore that nodes can move.

Consider a wireless ad hoc network. To elect a leader, any node in the network, called the source, can initiate an election by sending an ELECTION message to its immediate neighbors (i.e., the nodes in its range). When a node receives an ELECTION for the first time, it designates the sender as its parent, and subsequently sends out an ELECTION message to all its immediate neighbors, except for the parent. When a node receives an ELECTION message from a node other than its parent, it merely acknowledges the receipt.

When node R has designated node Q as its parent, it forwards the ELECTION message to its immediate neighbors (excluding Q) and waits for acknowledgments to come in before acknowledging the ELECTION message from Q. This waiting has an important consequence. First, note that neighbors that have already selected a parent will immediately respond to R. More specifically, if all neighbors already have a parent, R is a leaf node and will be able to report back to Q quickly. In doing so, it will also report information such as its battery lifetime and other resource capacities.

This information will later allow Q to compare R's capacities to that of other downstream nodes, and select the best eligible node for leadership. Of course, Q had sent an ELECTION message only because its own parent P had done so as well. In turn, when Q eventually acknowledges the ELECTION message previously sent by P, it will pass the most eligible node to P as well. In this way, the source will eventually get to know which node is best to be selected as leader, after which it will broadcast this information to all other nodes.

This process is illustrated in Figure 6.22. Nodes have been labeled a to j, along with their capacity. Node a initiates an election by broadcasting an ELECTION message to nodes b and j, as shown in Figure 6.22(b) After that step, ELECTION messages are propagated to all nodes, ending with the

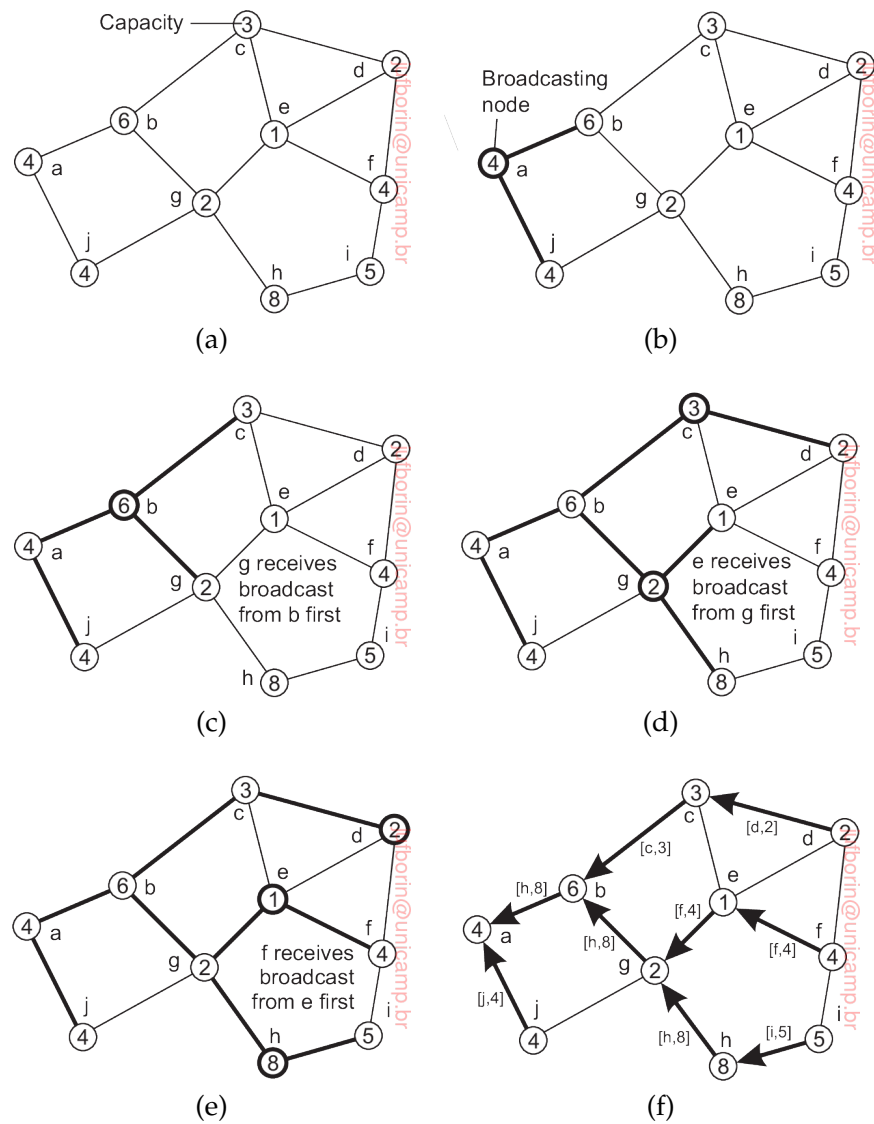


Figure 6.22: Election algorithm in a wireless network, with node *a* as the source. (a) Initial network. (b)–(e) The build-tree phase (last broadcast step by nodes *f* and *i* not shown). (f) Reporting of best node to source.

situation shown in Figure 6.22(e), where we have omitted the last broadcast by nodes *f* and *i*. From there on, each node reports to its parent the node with the best capacity, as shown in Figure 6.22(f). For example, when node *g* receives the acknowledgments from its children *e* and *h*, it will notice that *h* is the best node, propagating $[h, 8]$ to its own parent, node *b*. In the end, the source will note that *h* is the best leader and will broadcast this information to all other nodes.

When multiple elections are initiated, each node will decide to join only one election. To this end, each source tags its ELECTION message with a unique identifier. Nodes will participate only in the election with the highest identifier, stopping any running participation in other elections.

With some minor adjustments, the protocol can be shown to operate also when the network partitions, and when nodes join and leave. The details can be found in Vasudevan et al. [2004].

Elections in large-scale systems

Many leader-election algorithms apply to only relatively small distributed systems. Moreover, algorithms often concentrate on the selection of only a single node. There are situations when several nodes should actually be selected, such as in the case of **super peers** in peer-to-peer networks, which we discussed in Section 2.3. In this section, we concentrate specifically on the problem of selecting super peers.

The following requirements need to be met for super-peer selection (see also [Lo et al., 2005]):

1. Normal nodes should have low-latency access to super peers.
2. Super peers should be evenly distributed across the overlay network.
3. There should be a predefined portion of super peers relative to the total number of nodes in the overlay network.
4. Each super peer should not need to serve more than a fixed number of normal nodes.

Fortunately, these requirements are relatively easy to meet in most peer-to-peer systems, given the fact that the overlay network is either structured (as in DHT-based systems), or randomly unstructured (as, for example, can be realized with gossip-based solutions). Let us take a look at solutions proposed by Lo et al. [2005].

In the case of DHT-based systems, the basic idea is to reserve a fraction of the identifier space for super peers. In a DHT-based system, each node receives a random and uniformly assigned m -bit identifier. Now suppose we reserve the first (i.e., leftmost) k bits to identify super peers. For example, if we need N superpeers, then the first $\lceil \log_2(N) \rceil$ bits of any *key* can be used to identify these nodes.

To explain, assume we have a (small) Chord system with $m = 8$ and $k = 3$. When looking up the node responsible for a specific key K , we can first decide to route the lookup request to the node responsible for the pattern $K \wedge 11100000$ which is then treated as the superpeer.¹ Note that each node with identifier ID can check whether it is a super peer by looking up $ID \wedge 11100000$ to see if this request is routed to itself. Provided node identifiers are uniformly assigned to nodes, it can be seen that with a total of N nodes the number of super peers is, on average, equal to $2^{k-m}N$.

A completely different approach is based on positioning nodes in an m -dimensional geometric space. In this case, assume we need to place N super peers *evenly* throughout the overlay. The basic idea is simple: a total of N tokens are spread across N randomly chosen nodes. No node can hold more than one token. Each token represents a repelling force by which another token is inclined to move away. The net effect is that if all tokens exert the same repulsion force, they will move away from each other and spread themselves evenly in the geometric space.

This approach requires that nodes holding a token learn about other tokens. To this end, we can use a gossiping protocol by which a token's force is disseminated throughout the network. If a node discovers that the total forces that are acting on it exceed a threshold, it will move the token in the direction of the combined forces, as shown in Figure 6.23. When a token is held by a node for a given amount of time, that node will promote itself to superpeer.

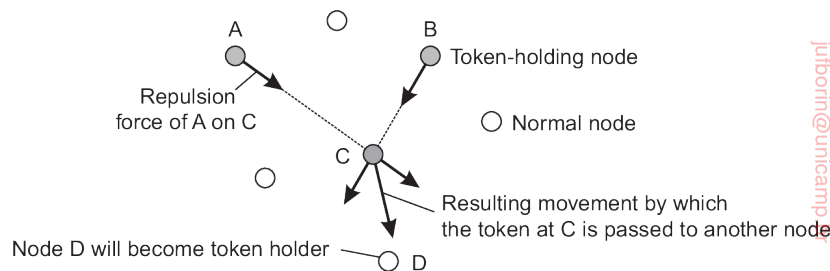


Figure 6.23: Moving tokens in a two-dimensional space using repulsion forces.

6.5 Location systems

When looking at very large distributed systems that are dispersed across a wide-area network, it is often necessary to take proximity into account. Just imagine a distributed system organized as an overlay network in which two processes are neighbors in the overlay network, but are actually placed far

¹We use the binary operator " $\wedge$ " to denote a bitwise and.